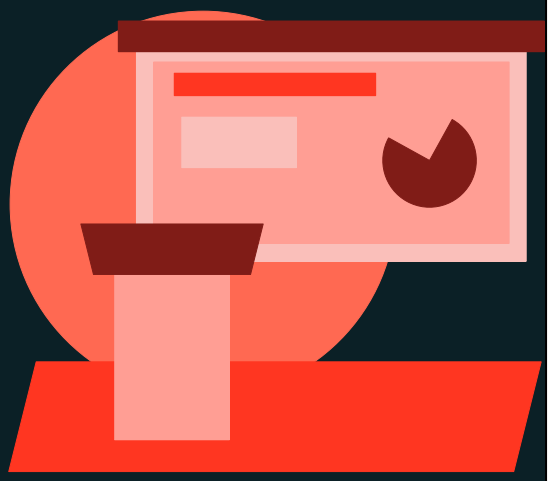
databricks

Creating and Scheduling Jobs

LECTURE

Common Task Configuration Options



© Databricks 2025. All rights reserved. Apache, Apache Spark, Spark, the Spark Logo, Apache Iceberg, Iceberg, and the Apache Iceberg logo are trademarks of the Apache Software Foundation.

Let's start with the common configuration options that you can apply to your tasks. These settings are key to building workflows that are not just automated, but also dynamic, context-aware, and easy to monitor.

We'll cover how to pass values into your tasks and how to set up alerts.

Common Task Configuration Options

Overview

Parameters & Dynamic Value References

Notification Alerts

Retries

All can be set at the Job level or at the individual Task level

© Databricks 2025. All rights reserved. Apache, Apache Spark, Spark, the Spark Logo, Apache Iceberg, Iceberg, and the Apache Iceberg logo are trademarks of the Apache Software Foundation.

Let me walk you through the three major categories of task configuration options:

Parameters & Dynamic Value References are the foundation of flexible workflows. These can only be set at the task level and allow you to create reusable, adaptable tasks that can behave differently based on input values or runtime context.

Retries are your first line of defense against transient failures. You can configure retry behavior at both job and task levels, allowing for different retry strategies depending on the criticality and expected failure patterns of different parts of your workflow.

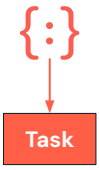
Notification Alerts keep your team informed and enable rapid response to issues. Like retries, these can be configured at both job and task levels, giving you granular control over who gets notified about what events.

Common Task Configuration Options

Parameter Overview

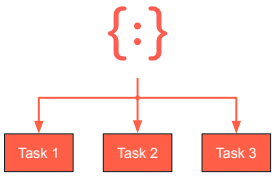
Task Parameters

A key-value pair defined at the **task level**



Job Parameters

A key-value pair defined at the **job level** and pushed down to all tasks



Job Parameters **override** Task Parameters when same **key** exists

© Databricks 2025. All rights reserved. Apache, Apache Spark, Spark, the Spark Logo, Apache Iceberg, Iceberg, and the Apache Iceberg logo are trademarks of the Apache Software Foundation.

Understanding the parameter hierarchy is crucial for effective job design:

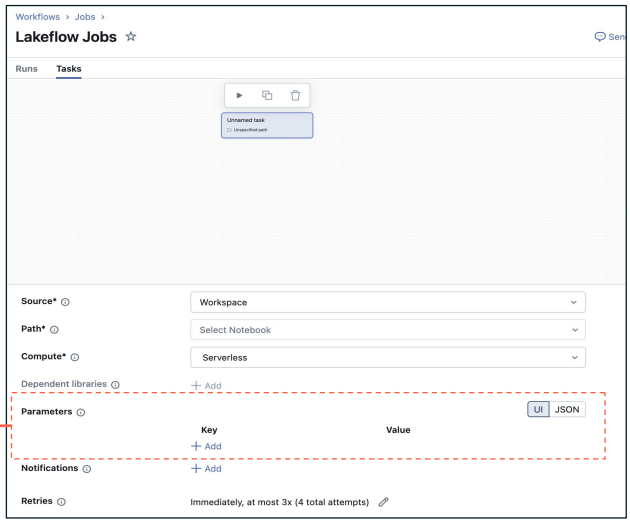
- Task Parameters are key-value pairs or JSON arrays defined at the individual task level. These are specific to each task and allow for fine-grained control over task behavior.
- Job Parameters are defined at the job level and automatically propagate to all tasks within that job. This creates a powerful inheritance model where you can set common defaults while still allowing task-specific overrides.

The precedence rule is critical to remember: Job parameters always override task parameters when the same key exists. This design pattern allows you to establish sensible defaults at the job level while maintaining the flexibility to customize individual tasks when needed.

Common Task Configuration Options

Task Parameter Overview

- **Task Parameters** are **key-value** pairs that lets you pass values into tasks
- Support **advanced orchestration** such as
  - Conditional execution
  - Looping
  - Passing context between tasks



© Databricks 2025. All rights reserved. Apache, Apache Spark, Spark, the Spark Logo, Apache Iceberg, Iceberg, and the Apache Iceberg logo are trademarks of the Apache Software Foundation.

Task parameters are far more than simple configuration values - they're the building blocks of intelligent workflows. These key-value pairs enable sophisticated orchestration patterns:

- Conditional Execution: Use parameters to control which branches of your workflow execute based on data conditions, environment settings, or business rules.
- Looping: Parameters can control iteration counts, define arrays for for-each loops, and manage complex processing scenarios.
- Context Passing: Share information between tasks by setting parameters that downstream tasks can read, creating a data flow alongside your control flow.

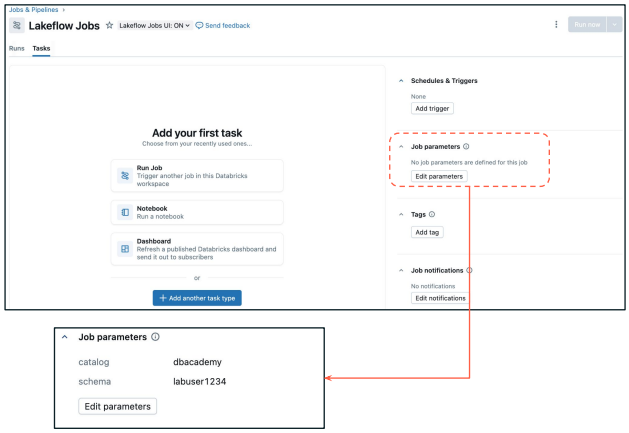
The real power comes from combining parameters with dynamic value references, allowing your workflows to adapt intelligently to changing conditions and data characteristics.

Common Task Configuration Options

Job Parameter Overview

Job Parameters are **key-value pairs** defined at the **job level** that provide default values for the entire workflow.

- Applied to **all tasks** in the job automatically
- **Override task parameters** with the same key name
- Can be **overridden at runtime** when triggering job runs



© Databricks 2025. All rights reserved. Apache, Apache Spark, Spark, the Spark Logo, Apache Iceberg, Iceberg, and the Apache Iceberg logo are trademarks of the Apache Software Foundation.

Job parameters serve as the foundation for consistent, maintainable workflows. They're key-value pairs that provide default values for your entire workflow, ensuring consistency across all tasks.

Here's what makes them powerful:

- Automatic Application: Every task in the job automatically receives these parameters, eliminating the need to manually configure common settings across multiple tasks.
- Override Capability: Tasks can still define their own parameters with the same key names, but job parameters take precedence, giving you centralized control.
- Runtime Flexibility: You can override job parameters when triggering job runs, allowing the same job definition to behave differently for different scenarios - perhaps different environments, date ranges, or processing modes.

Common Task Configuration Options

Setting and Accessing Parameters (UI)

Setting Parameter Values (UI)

Accessing Parameter Values in a Task

```
catalog = dbutils.widgets.get("catalog_name")

schema = dbutils.widgets.get("schema_name")
```

Setting Job Parameters:

- In your **job**, go to the **Parameters section**, and add key–value pairs at the job level.

Setting Task Parameters:

- Go to the **specific task in the job**,
- Go to **parameters** within the **task configuration** (alongside task name, type, and path)
- Add the key–value pairs

- Access both **task and job parameters** in notebook tasks with **`dbutils.widgets.get`**
- Retrieval methods vary by **task type** (e.g., notebook vs. SQL vs. Python wheel)

© Databricks 2025. All rights reserved. Apache, Apache Spark, Spark, the Spark Logo, Apache Iceberg, Iceberg, and the Apache Iceberg logo are trademarks of the Apache Software Foundation.

Let's look at the practical implementation of parameters:

- Setting Job Parameters: Navigate to your job's Parameters section and add key-value pairs at the job level. These become available to all tasks automatically. Common examples include catalog names, schema names, environment settings, and processing dates.
- Setting Task Parameters: Within each task's configuration (found alongside task name, type, and path settings), add task-specific key-value pairs. These are perfect for task-specific paths, processing options, or override values.
- Retrieving in Notebook Tasks: Use `dbutils.widgets.get("parameter_name")` to access both job and task parameters. The system automatically handles the precedence - if both job and task parameters exist with the same key, you'll get the job parameter value
- Language-Specific Retrieval: Remember that parameter retrieval methods vary by task type. SQL tasks access parameters differently than Python wheel tasks or JAR tasks. Always check the documentation for your specific task type.

Common Task Configuration Options

Dynamically Setting and Accessing Task Parameters

Dynamically Set Task Values with Code

```
dbutils.jobs.taskValues.set(
                key = "catalog_name",
                value = "dbacademy")
```

Task Values are **dynamic key-value pairs** that tasks create and share during **workflow execution**.

Accessing a Parameter from another Task

```
dbutils.jobs.taskValues.get(
            taskKey="task-name",
            key="catalog_name")
```

You Reference specific upstream task parameter values into **downstream tasks**

© Databricks 2025. All rights reserved. Apache, Apache Spark, Spark, the Spark Logo, Apache Iceberg, Iceberg, and the Apache Iceberg logo are trademarks of the Apache Software Foundation.

Task Values represent a more advanced pattern than parameters - they're computed at runtime and enable dynamic communication between tasks:

Setting Task Values: Use `dbutils.jobs.taskValues.set(key="computed_result", value="some_calculated_value")` to store results that other tasks need. This might include record counts, processing status, file paths, or any computed information.

Retrieving Task Values: Use `dbutils.jobs.taskValues.get(taskKey="upstream-task-name", key="computed_result")` to access values from specific upstream tasks. This creates explicit dependencies between tasks based on data, not just execution order.

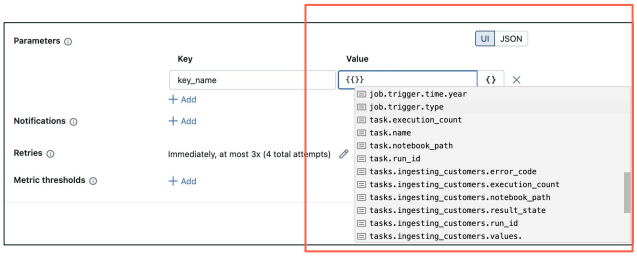
Use Cases: Task values are perfect for sharing computed results like record counts for data quality checks, file paths for dynamically generated outputs, processing statistics for monitoring, or decision criteria for conditional logic.

Execution Context: Unlike parameters which are set before execution, task values are created during execution, making them ideal for decisions that depend on processing results.

Common Task Configuration Options

Dynamic Value References

- **Dynamic Value References** allow referencing values at **runtime** from the job and task context (e.g., task outputs, parameters, retask values, registered task values, run metadata).
- These references use `{{ }}` notation and allow workflows to adapt to different execution.
- Common examples:
  - `{{job.start_time.day }}` - For accessing the day
  - `{{task.name}}` - For Accessing Task Name
  - `{{tasks.<task-name>.values.my_value }}` For accessing output from an upstream task



© Databricks 2025. All rights reserved. Apache, Apache Spark, Spark, the Spark Logo, Apache Iceberg, Iceberg, and the Apache Iceberg logo are trademarks of the Apache Software Foundation.

Dynamic Value References using `{{ }}` notation unlock powerful runtime capabilities that make workflows truly adaptive:

Job Context References:

- `{{job.start_time.day}}` - Access execution timing for date-based processing
- `{{job.run_id}}` - Unique identifier for tracking and logging
- `{{job.parameters.environment}}` - Access job-level parameters dynamically

Task Context References:

- `{{task.name}}` - Useful for logging and dynamic path generation
- `{{task.retry_count}}` - Track retry attempts for debugging

Inter-Task Communication:

- `{{tasks.data-validation.values.record_count}}` - Access computed results from upstream tasks
- `{{tasks.file-processor.values.output_path}}` - Use dynamic paths generated by other tasks

Advanced Patterns: These references enable workflows that adapt to different execution environments, process varying data volumes, and make intelligent decisions based on upstream results.

Common Task Configuration Options

Notification Alerts

Notification Configurations

Job Level Notifications

- A notification alert is sent after **successful completion of the job**
- This setting can be adjusted in the Job Details section (right-side pane).

Task Level Notifications

- A notification alert will be sent when the **task is successfully completed**
- This notification setting can be customized at the task level.

© Databricks 2025. All rights reserved. Apache, Apache Spark, Spark, the Spark Logo, Apache Iceberg, Iceberg, and the Apache Iceberg logo are trademarks of the Apache Software Foundation.

Notification alerts form a critical part of operational excellence, and understanding the configuration levels helps you build effective alerting strategies:

Job Level Notifications: Configure these in the Job Details section's right-side pane. Job-level alerts are sent after the entire job completes successfully. This is perfect for stakeholders who need to know when complete workflows finish, such as business users waiting for daily reports or downstream systems that depend on your job's outputs.

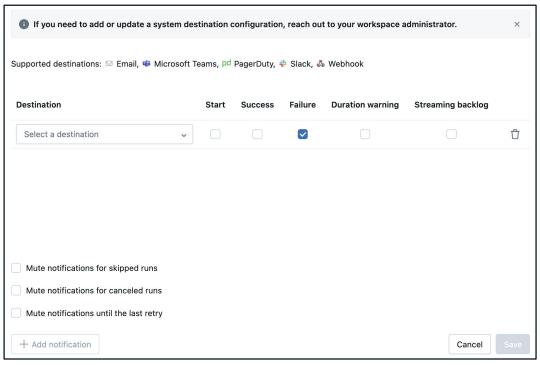
Task Level Notifications: Each task can have its own notification configuration, allowing granular alerting strategies. This is essential when different tasks have different stakeholders or when certain tasks are more critical than others. For example, you might want immediate alerts for data validation failures but only summary notifications for routine cleanup tasks.

Strategic Considerations: Design your notification strategy based on operational needs, not technical convenience. Consider who needs to know what, when they need to know it, and what actions they can take based on the notification.

Common Task Configuration Options

Notification Alerts

- **Multiple** supporting destinations as Emails, Teams, pagerDuty,Slack and Webhook
- **Each task in a Job** could be configured **differently** to send notification
- Jobs triggers **notifications** when the task begins, completes or fails
- These notification could also be triggered in the case of **late Jobs:**
  - For duration threshold warning/timeout alerts
  - Streaming backlog(Issue with getting streaming data)
- Webhook allows custom integration with third-party API services



© Databricks 2025. All rights reserved. Apache, Apache Spark, Spark, the Spark Logo, Apache Iceberg, Iceberg, and the Apache Iceberg logo are trademarks of the Apache Software Foundation.

Modern production environments require sophisticated notification strategies:

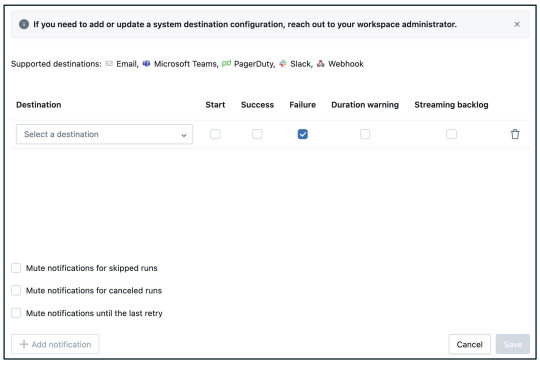
Multiple Destinations: Support for Emails, Microsoft Teams, PagerDuty, Slack, and Webhooks means you can integrate with your existing operational tools and communication patterns. Different teams might prefer different channels - developers might want Slack notifications while operations teams prefer PagerDuty integration.

Per-Task Customization: Each task in a job can have completely different notification configurations. Your data ingestion tasks might send alerts to the data engineering team, while your reporting tasks notify business stakeholders.

Common Task Configuration Options

Notification Alerts

- **Multiple** supporting destinations as Emails, Teams, pagerDuty,Slack and Webhook
- **Each task in a Job** could be configured **differently** to send notification
- Jobs triggers **notifications** when the task begins, completes or fails
- These notification could also be triggered in the case of **late Jobs:**
  - For duration threshold warning/timeout alerts
  - Streaming backlog(Issue with getting streaming data)
- Webhook allows custom integration with third-party API services



© Databricks 2025. All rights reserved. Apache, Apache Spark, Spark, the Spark Logo, Apache Iceberg, Iceberg, and the Apache Iceberg logo are trademarks of the Apache Software Foundation.

Advanced Trigger Conditions: Beyond simple success/failure, you can configure notifications for:

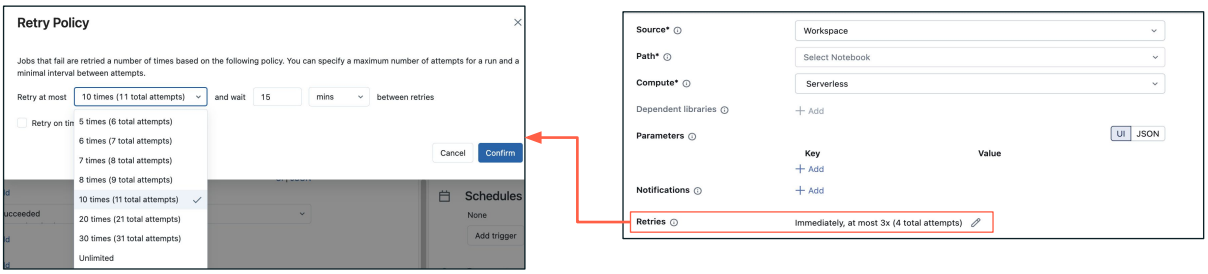
- Late Jobs: Duration threshold warnings and timeout alerts help you catch performance degradation early
- Streaming Backlog: Critical for streaming workloads where falling behind can cascade into larger issues
- Custom Conditions: Webhook integrations allow complex custom logic for notification decisions

Lifecycle Notifications: Jobs can trigger notifications when tasks begin (useful for long-running processes), complete successfully (confirmation of completion), or fail (immediate response required).

Common Task Configuration Options

Retry Policy

A policy that determines when and how many times failed runs are **retried**



© Databricks 2025. All rights reserved. Apache, Apache Spark, Spark, the Spark Logo, Apache Iceberg, Iceberg, and the Apache Iceberg logo are trademarks of the Apache Software Foundation.

A well-designed retry policy is essential for resilient workflows. The policy determines not just how many times to retry, but under what conditions and with what timing patterns.

Consider factors like:

- Failure Type: Transient network issues might warrant immediate retries, while data quality issues might not
- Resource Impact: Retrying resource-intensive tasks too aggressively can cause cluster resource contention
- Downstream Dependencies: Failed tasks might impact other workflows, making retry timing critical
- Business SLA: Some processes have strict timing requirements that limit retry windows

© Databricks 2025. All rights reserved. Apache, Apache Spark, Spark, the Spark Logo, Apache Iceberg, Iceberg, and the Apache Iceberg logo are trademarks of the Apache Software Foundation.

Thank you for completing this lesson and continuing your journey to develop your skills with us.